

Rapport de Projet ChenYI-Tech

Description de l'Équipe et du Sujet :

Nous sommes deux étudiantes en première année de classe préparatoire mathématiques-informatique à CY TECH. Dans le cadre de notre projet, nous avons développé une application en langage C appelée Refuge ChenYI-Tech, destinée à la gestion des animaux d'un refuge. L'objectif était de concevoir un programme capable d'enregistrer les animaux, de les rechercher, de gérer les adoptions, de calculer leurs besoins en nourriture et de générer un inventaire.

Planning et Chronologie :

Après le cours magistral présentant le projet, nous avons commencé à vraiment nous y intéresser. Étant seulement deux sur le projet, nous avons rapidement pris l'habitude de nous organiser à deux, en échangeant régulièrement sur WhatsApp, notamment via des appels en visio pour avancer ensemble. Plutôt que de tout faire d'un coup, on a choisi de progresser étape par étape en se fixant des objectifs clairs pour chaque fonctionnalité. Cette méthode nous a permis de mieux gérer notre temps, de suivre notre avancement, et d'ajuster notre travail en fonction des difficultés rencontrées.

Phase de Test et de Validation :

On a toujours accordé beaucoup d'importance au bon fonctionnement de notre application. À chaque fois qu'on terminait une fonctionnalité, on la testait tout de suite pour s'assurer qu'elle marchait comme prévu. On faisait aussi des tests plus globaux pour voir si tout s'enchaînait bien entre les différentes parties du programme, notamment avec la gestion des fichiers. Ces phases de test nous ont permis de repérer pas mal de petits bugs, et parfois même de repenser certaines parties de notre code pour le rendre plus fiable et plus clair.

Problèmes Rencontrés et Solutions Apportées :

Le développement en langage C, combiné à la gestion de données persistantes sur fichier texte, a présenté plusieurs défis techniques :

1. Gestion des Fichiers et Extraction des Données

Problème : La lecture des informations structurées depuis le fichier animaux/animaux.txt, formatées avec des points-virgules comme séparateurs, nous a demandé une grande rigueur. Certains champs comme les noms ou les commentaires pouvaient contenir des espaces, ce qui compliquait l'utilisation des fonctions classiques comme scanf. De plus, une ligne mal formée pouvait entraîner des erreurs de lecture difficiles à diagnostiquer.

Solution : Nous avons choisi d'utiliser fgets pour lire chaque ligne complète, puis sscanf avec des formats précis comme %[^\n;] pour extraire les champs. À chaque lecture, nous avons vérifié que le nombre de champs lus correspondait bien à ce qui était attendu, ce qui nous a permis d'identifier et de corriger rapidement les éventuelles erreurs de format.

2. Validation des Entrées Utilisateur

Problème : L'un des défis fréquents a été de gérer correctement les saisies utilisateur. Le moindre caractère inattendu pouvait provoquer un dysfonctionnement, notamment à cause du comportement de scanf et de son interaction avec le buffer d'entrée.

Solution : Nous avons mis en place des boucles de validation pour toutes les saisies importantes. L'utilisateur est invité à recommencer tant que la donnée n'est pas valide. Pour éviter les effets indésirables sur les lectures suivantes, nous avons également vidé le buffer manuellement à chaque fois que cela s'avérait nécessaire.

Implémentation de l'Adoption (Suppression) :

Problème : Réaliser la suppression d'une entrée spécifique dans un fichier texte sans devoir gérer manuellement la réécriture du fichier complet. Les fonctions de manipulation de fichiers en C ne permettent pas de supprimer directement une ligne au milieu d'un fichier plat.

Solution : Nous avons implémenté la méthode standard pour ce cas : lire le fichier original, écrire toutes les lignes sauf celle correspondant à l'animal à adopter dans un fichier temporaire, puis supprimer le fichier original et renommer le fichier temporaire pour le remplacer.

Organisation et Flux de Travail :

Pour structurer notre travail, nous avons principalement utilisé WhatsApp pour nos échanges quotidiens. Cela nous a permis de discuter rapidement des progrès et de résoudre des problèmes en temps réel. Afin de gérer les différentes versions du programme et intégrer nos contributions respectives, nous avons utilisé Git. Ce système de contrôle de version a facilité la collaboration, en assurant une gestion claire et structurée du code tout au long du projet.

Réflexions et Apprentissages :

Ce projet a été une super occasion d'apprendre. Il nous a permis de renforcer nos compétences en C et d'acquérir une expérience concrète en gestion de projet. On a beaucoup travaillé sur la manipulation des fichiers, ce qui nous a aidées à mieux comprendre leur fonctionnement et comment les utiliser dans un programme. La gestion des entrées/sorties et la validation des données ont aussi été des parties importantes du projet. Structurer notre code en modules et gérer les interactions entre les différentes parties du programme ont été des étapes cruciales pour avancer. Les difficultés techniques qu'on a rencontrées nous ont montrées l'importance d'être méthodiques et de bien faire attention aux détails, ce qui est essentiel pour réussir en développement C

Conclusion :

Pour conclure, ce projet a été une super expérience en développement logiciel, surtout en langage C. En surmontant les différents défis techniques, nous avons non seulement amélioré nos compétences en programmation, mais aussi appris à mieux organiser notre travail et à résoudre les problèmes de façon plus méthodique. La gestion des fichiers, la validation des données et la structuration du code en modules ont aussi été des points clés que nous avons approfondis. Nous sommes contentes d'avoir réussi à créer une application qui fonctionne bien et est solide. Ce projet nous a montré à quel point il est important d'être rigoureuses et organisées, et nous avons maintenant une meilleure idée de ce qu'il faut pour gérer un projet informatique.